Article PDF Available

Refinement for User Interface Designs

November 2009 · Formal Aspects of Computing 21(6):589-612
DOI:10.1007/s00165-008-0095-2
Source · DBLP

Authors:

Judy Bowen
The University of Waikato

Steve Reeves
The University of Waikato

Citations (21) References (33) Figures (3)

Abstract and Figures

Formal approaches to software development require that we correctly describe (or specify) systems in order to prove properties about our proposed solution prior to building it. We must then follow a rigorous process to transform our specification into an implementation to ensure that the properties we have proved are retained. Different transformation, or refinement, methods exist for different formal methods, but they all seek to ensure that we can guide the transformation in a way which preserves the desired properties of the system. Refinement methods also allow us to subsequently compare two systems to see if a refinement relation exists between the two. When we design and build the user interfaces of our systems we are similarly keen to ensure that they have certain properties before we build them. For example, do they satisfy the requirements of the user? Are they designed with known good design principles and usability considerations in mind? Are they correct in terms of the overall system specification? However, when we come to implement our interface designs we do not have a defined process to follow which ensures that we maintain these properties as we transform the design into code. Instead, we rely on our judgement and belief that we are doing the right thing and subsequent user testing to ensure that our final solution remains useable and satisfactory. We suggest an alternative approach, which is to define a refinement process for user interfaces which will allow us to maintain the same rigorous standards we apply to the rest of the system when we implement our user interface designs.

Design UI A and UI C

Design UI C 5

Sequential Charts for UI…

Electronic Notes in Theoretical Computer Science 208 (2008) 5–22
www.elsevier.com/locate/entcs

Refinement for User Interface Designs

Judy Bowen[1] and Steve Reeves[2]

Department of Computer Science
University of Waikato
Hamilton, New Zealand

Abstract

Formal approaches to software development require that we correctly describe (or specify) systems in order to prove properties about our proposed solution prior to building it. We must then follow a rigorous process to transform our specification into an implementation to ensure that the properties we have proved are retained. Different transformation, or refinement, methods exist for different formal methods, but they all seek to ensure that we can guide the transformation in a way which preserves the desired properties of the system. Refinement methods also allow us to subsequently compare two systems to see if a refinement relation exists between the two. When we design and build the user interfaces of our systems we are similarly keen to ensure that they have certain properties before we build them. For example, do they satisfy the requirements of the user? Are they designed with known good design principles and usability considerations in mind? Are they correct in terms of the overall system specification? However, when we come to implement our interface designs we do not have a defined process to follow which ensures that we maintain these properties as we transform the design into code. Instead, we rely on our judgement and belief that we are doing the right thing and subsequent user testing to ensure that our final solution remains useable and satisfactory. We suggest an alternative approach, which is to define a refinement process for user interfaces which will allow us to maintain the same rigorous standards we apply to the rest of the system when we implement our user interface designs.

Keywords: Refinement, user interface, formal methods, user-centred design.

1 Introduction

User-centred design (UCD) and an iterative approach to building user interfaces (UIs) allows us to keep users' requirements central to our design and ensure that we consider their feedback as we amend that design. At the same time we can ensure that our interface designs reflect the requirements of both the user and the overall system by incorporating them into a formal design process. We have previously derived a way of integrating UI designs into a formal software development process by way of formal models, [5] and [3], which ensure that the UI and system designers are working towards the same end goal.

[1] Email: jab34@cs.waikato.ac.nz
[2] Email: stever@cs.waikato.ac.nz

doi:10.1016/j.entcs.2008.03.104

design becomes an implementation we preserve the important properties we have considered during the design stage.

Our system development approach is one where different parts of the system, such as the UI and underlying application logic, are considered separately. This means that as well as relating them during design stages and ensuring that each part is correctly implemented, we must also ensure that the combination of the parts also remains correct.

This then is our motivation for investigating refinement for UIs, to make sure that what we implement is what we intended. We want all of the guarantees of correctness for the UI that we have for the rest of the system. We therefore need some structured and formal way of transforming our designs into implemented UIs, that is we need a refinement process.

One consideration is how we go about generating the code for our UIs. Many software development applications, (such as Visual Studio [12] or Eclipse [7]) utilise 'drag and drop' toolboxes of UI elements which allow quick development of UI layouts and widgets and allow us to delay the programming of behaviour of these widgets. It may be that these UIs are used as interim iterative prototypes and that subsequently we use some other target language for our final implementation, or it may be that we develop these prototypes into fully working UIs and systems within these development environments. In either case there are different stages of development where changes will be made to the UI as we get closer and closer to our end product.

This reflects the incremental approach to system implementation we refer to as *stepwise* refinement [15]. Irrespective of what the intermediate steps are (paper designs, mock-ups, partially functioning UIs, full implementations *etc.*) we want a way of maintaining correctness. This approach to UI refinement is different from that proposed in works such as [6] and [11] in that we are not starting from a single system specification which formalises the UI behaviour as one part of the system, but rather extending traditional UI design methods in a non-traditional, formal manner.

This paper consists of two parts. We start by examining some traditional notions of refinement to see how conceptually they may be applied to UI designs. We will use this as the basis for an informal description of UI refinement and show via some small examples how this may be applied. In the second part we will look at how refinement might be formalised, and introduce the idea of $Sys \parallel UI$ composition using the μCharts language. We will conclude with a discussion about monotonicity and its importance and future work.

2 Refinement

Refinement is a formal process which allows us to transform one system into another in a manner which ensures that required properties of the original system are preserved. By system we mean any description at any level of abstraction from specification to implementation, or anything in between. Refinement rules can be used to guide the transformation from one system to another, and can also be used to compare two systems to see if one is a correct refinement of another.

Different refinement methods exist for different formal languages, but generally they can be categorised by a common understanding of what the underlying principles of refinement are. We are interested in how these general principles may apply

ciples of refinement individually to consider their suitability as principles for UI refinement.

2.1 Principle of Substitutivity

The principle of substitutivity states that it is acceptable to replace one program by another provided it is impossible for a user to observe that the substitution has taken place.

Usually when we talk about substitution we are considering observable behaviours of systems in terms of either input/output traces, or interaction with other parts of the system, *i.e.* behaviour devoid of any notion of visual appearance or cognitive awareness of differences. For UIs, however, such visual and cognitive differences are important; if we substitute one UI for another and they are visually different then the user (who in this case is a real person and not some computer process) will be able to tell that the substitution has taken place. Rather than considering substitutivity we consider the principle behind this concept, namely that of considering programs as contracts.

In [13] Morgan states:

> "A program has two roles: it describes what one person wants, and what another person (or computer) must do."

In this context, refinement must always provide the customer with the ability to do at least the same things they could previously, or more. That is, we can replace one thing with another as long as the customer gets at least what they had before or better (for our purposes by customer we may mean either the end-user or some member of the design team). This is similar to the principle of substitutivity in that it gives conditions under which we can replace one thing with another, but the requirement here is on maintaining utility rather than hiding the substitution. We will refer to this as satisfying *contractual utility*.

As well as the behaviour/functionality of the UI we will also want to consider usability aspects of the UIs, regardless of behaviour; if the replacement UI is perceived to be harder to use than the original then the customer will not be satisfied.

dimensions. The first dimension is the type of model and where it falls on the scale of formality-from fully formal approaches, such as the use of formal specifications, verification, refinement, etc. [Blandford et al., 2011 , Bowen & Reeves, 2007 , Harrison et al., 2017, to the informal methods of usercentred design and more 'agile' design approaches [Blandford et al., 2010] as well as everything in between [Calvary et al., 2001, Seffah et al., 2009. The second dimension is how, and where, the models are used: at the design stage before any implementation occurs; as part of the final testing and sign-off of the implemented solution; or throughout the design process. ...

Using Task Models to Understand the Intersection of Numeracy Skills and Technical Competence With Medical Device Design

Article

May 2021

Judy Bowen · Diana Coben

View Show abstract

... Bowen and Reeves [22] present a way of applying the specification language µCharts and refinement processes to UI designs. They use presentation models to compare two UI designs and if these UI maintain usability. ...

A Survey of Papers from Formal Methods for Interactive Systems (FMIS) Workshops

Chapter

Aug 2020

Pascal Béger · Sébastien Leriche · Daniel Prun

View Show abstract

... This model is gradually refined using details about how specific functionalities are implemented. Bowen and Reeves [27] have presented a similar refinement approach for user interface design. Their work specifically targets the design of user interface layouts. ...

Verification of User Interface Software: The Example of Use-Related Safety Requirements and Programmable Medical Devices

Article Full-text available

Jul 2017

Michael Harrison · Paolo Masci · José Creissac Campos · Paul Curzon

View Show abstract

... In [5] a formal approach to verify if a UI is a refinement of another UI is proposed, by verifying functional equivalence, for instance. This approach verifies if a UI provides at least as much functionalities as another UI. ...

Plasticity of user interfaces

Conference Paper

Jun 2015

Raquel Oliveira · Sophie Dupuy-Chessa · Gaelle Calvary

View Show abstract

Interaction Modelling for IoT

Conference Paper

Dec 2021

Jessica Turner · Judy Bowen · Nikki van Zandwijk

View

Software Technologies: Applications and Foundations - STAF 2018 Collocated Workshops, June 25-29, 2018, Revised Selected Papers, Toulouse, France, 25/06/2018 - 29/06/2018

Book

Jan 2018

Manuel Mazzara · Iulian Ober · Gwen Salaun

A Formal Machine–Learning Approach to Generating Human–Machine Interfaces From Task Models

Article

May 2017

Meng Li · Jiajun Wei · Xi Zheng · M.L. Bolton

View Show abstract

Mandatory and Potential Choice: Comparing Event-B and STAIRS

Chapter Full-text available

May 2016

Atle Refsdal · Ragnhild Kobro Runde · Ketil Stølen

View Show abstract

Creating models of interactive systems with the support of lightweight reverse-engineering tools

Conference Paper

Jun 2015

Judy Bowen

View Show abstract

Equivalence checking for comparing user interfaces

Conference Paper

Jun 2015

Raquel Oliveira · Sophie Dupuy-Chessa · Gaelle Calvary

View Show abstract